



Workshop Guide

Surf API from Zero

Build your own ocean conditions web app with Claude Code

Agüita House · Sustainability Lab

June 11, 2026 — Las Palmas de Gran Canaria

Welcome

Today you'll build a real web application: a live dashboard of ocean conditions at **La Cícer Beach** (tides, wind, and waves). You don't need to know how to code. You only need to read, copy a few prompts, and watch the browser.

What you'll see at the end:



Finished app

What you'll learn:

- How to think about a software project (in small pieces).
- How to ask **Claude Code** to write the code for you.
- How to see your changes live in the browser.
- How to make the project your own: change the beach, the colors, add more data.

You do NOT need: coding experience, technical English, or anything about networks. Just follow the steps.

Estimated time: 2 hours.

Setup checklist

Before we start, you need these on your laptop. If you already have them, skip ahead.

Tool	How to install	How to check it works
VS Code	Download from https://code.visualstudio.com/ and open it.	A dark window opens with a sidebar menu.
Python 3	macOS and Linux already have it. On Windows, download from https://python.org .	Open the terminal and type <code>python3 --version</code> → it should reply with something like <code>Python 3.x.x .</code>
Claude Code	Follow the instructions at https://claude.com/claude-code	In the terminal type <code>claude --version</code> and it should reply with a number.
A Claude account	Sign up for free at https://claude.ai/	The first time you open Claude Code, it'll ask you to log in.

Five terminal commands you'll use all the time:

- `cd folder` → go into a folder.
 - `cd ..` → go up one folder.
 - `ls` → list what's in the current folder.
 - `pwd` → show which folder I'm in.
 - Up arrow ↑ → repeat the last command.
 - `Ctrl + C` → stop something that's running in the terminal.
-

Meet Claude Code

Claude Code is like having an expert programmer sitting next to you. You speak to it in plain language, and it writes the code for you.

How to open it:

1. Open VS Code.
2. In the top menu, choose **Terminal** → **New Terminal**.
3. In that terminal, type `claude` and press Enter.
4. A box appears where you can type. That's your line to Claude.

Three examples of how to talk to it:

PROMPT FOR CLAUDE CODE

Example 1:

Read the structure of this project and explain what's in each file.

PROMPT FOR CLAUDE CODE

Example 2:

Create a file called `hello.html` with a big title that says "Hello world".

PROMPT FOR CLAUDE CODE

Example 3:

The browser console shows an error that says "Failed to fetch". Read it, find it in my code, and fix it.

The golden rule: be specific. Instead of saying *"make a nice page"*, say *"make a page with a dark blue background, a centered white title, and three light blue buttons"*.

If the result isn't what you wanted, no problem — just say *"that's not it, I'd prefer..."* and Claude will change it.

The project we're building

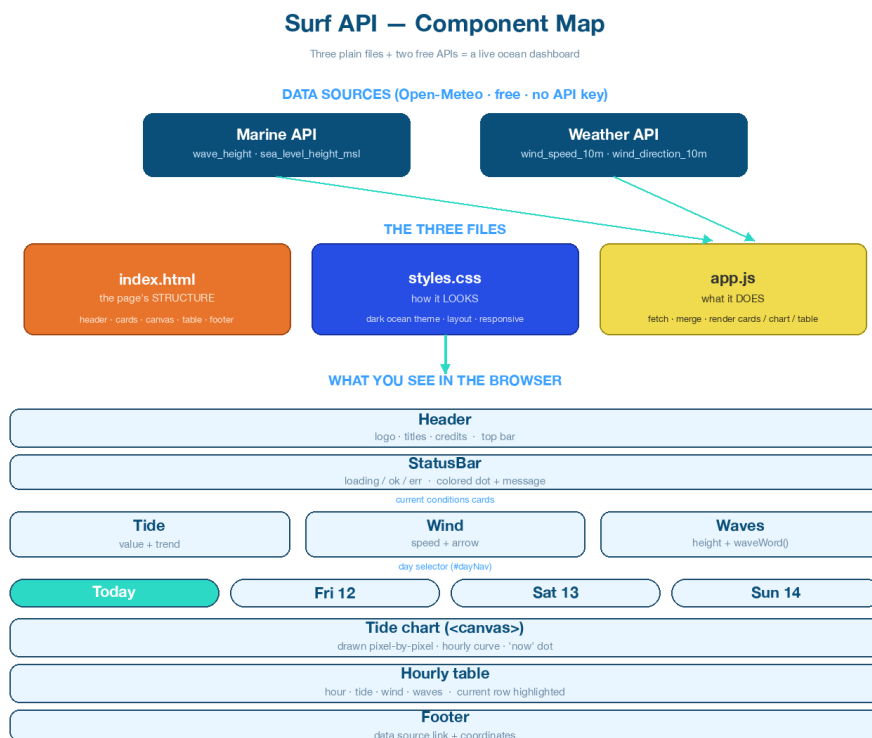
Surf API is a web page that shows, live, the ocean conditions at La Cícer:

- The **tide** right now and whether it's rising or falling.
- The **wind** (speed and direction).
- The **waves** (significant height).
- A **chart** of how the tide changes hour by hour.
- A **table** with the hourly forecast, up to 4 days ahead.

The tech (in one sentence): a web page made of three plain files — **HTML** (structure), **CSS** (styling), and **JavaScript** (logic) — that asks **Open-Meteo** for data. Open-Meteo is a free service that provides weather and marine data with no API key required.

No frameworks. No build step. No dependencies. Just three files that any web browser can open.

The project's pieces:



Component map

The three files:

1. `index.html` — the page's structure (header, status bar, three cards, day buttons, chart, table, footer).
2. `styles.css` — what it looks like (dark “ocean” theme, layout, animations).
3. `app.js` — what it does (fetches the data, fills in the cards, draws the chart, renders the table).

Where the data comes from:

- Tides and waves: `marine-api.open-meteo.com`
- Wind: `api.open-meteo.com`

Both URLs are free and require no signup.

Step 1 — Create the project

Goal

A folder called `surf-api` with three empty files (`index.html` , `styles.css` , `app.js`) being served by a tiny local web server, and you can open it in the browser.

Do it

Open the terminal in the folder where you keep your projects (e.g. `~/Documents`):

```
cd ~/Documents
mkdir surf-api
cd surf-api
touch index.html styles.css app.js
```

(On Windows, replace `touch index.html styles.css app.js` with `type nul > index.html` then repeat for the other two.)

Now start a local web server in this folder:

```
python3 -m http.server 8000
```

What you should see

The terminal prints something like `Serving HTTP on :: port 8000` . Open <http://localhost:8000> in your browser → you'll see a mostly blank page (because the files are empty).

Try it yourself

Leave the terminal running. From now on, any change we save to these files will show up when you refresh the browser tab.

Step 2 — Open the project with Claude Code

Goal

Get Claude Code to know your project and help you from the inside.

Do it

1. In VS Code: **File** → **Open Folder** → `surf-api`.
2. Open a *new* terminal in VS Code (**Terminal** → **New Terminal**) — keep the one running the server alone.
3. Type `c laude` and press Enter.

When it's ready, ask it:

PROMPT FOR CLAUDE CODE

Read the structure of this project and explain to me, in plain English, what's in this folder.

What you should see

Claude will tell you there are three empty files and explain what each one will be for. Perfect — that's our starting point.

Try it yourself

Ask it:

PROMPT FOR CLAUDE CODE

Write a single line of HTML in `index.html` that shows "Hello, ocean!" as a big title.

Save the file, refresh the browser tab, and you'll see the text.

Step 3 — Build the HTML structure

Goal

A complete `index.html` with all the placeholders for our app: header, status bar, three condition cards, day buttons, tide chart canvas, hourly table, and footer.

Do it

PROMPT FOR CLAUDE CODE

Replace the contents of `index.html` with a complete HTML5 document containing:

1. A `<head>` with charset, viewport meta, the page title “Agüita House Sustainability Lab – La Cícer Beach Conditions”, a description meta tag, `<link rel="preconnect">` to both `marine-api.open-meteo.com` and `api.open-meteo.com`, and a `<link>` to `styles.css`.
2. A `<header class="site-header">` with two parts:
 - A `<div class="brand">` containing a 🌊 emoji in `<div class="brand-mark">` and a `<div class="brand-text">` with `<h1>Agüita House` `<span>Sustainability Lab</span></h1>` and `<p class="brand-sub">La Cícer Beach · Las Palmas de Gran Canaria</p>`.
 - A `<div class="credits">` with “by **Nicola Gasparro** & **Vicente Matus**” and “Sponsored by **IDeTIC · ULPGC**”.
3. A `<main class="container">` containing:
 - `<section class="status-bar">` with a `<span class="dot" id="statusDot">` and a `<span id="statusText">Loading live data...</span>`.
 - `<section class="cards">` with three `<article class="card">` elements for **Tide** (id `curTide`, trend id `curTideTrend`), **Wind** (id `curWind`, direction id `curWindDir`, arrow id `curWindArrow`), and **Waves** (id `curWave`, subtitle id `curWaveSub`). Each card has an `<h2>`, a big number, a unit, and a subtitle line.
 - `<nav class="day-nav" id="dayNav">` (empty – JavaScript fills it).
 - `<section class="panel">` with `<h2>Tide by the hour</h2>` and a `<canvas id="tideChart" height="220">`.
 - `<section class="panel">` with `<h2>Hourly forecast</h2>` and a `<table class="hourly">` with `<thead>` (columns: Hour, Tide (m), Wind, Wave (m)) and an empty `<tbody id="hourlyBody">`.
 - A `<footer class="site-footer">` with a paragraph linking to Open-Meteo and a `<p class="coords" id="coords">` (empty – JavaScript fills it).
4. Finally, a `<script src="app.js"></script>` just before `</body>`.

What you should see

Refresh the browser. You'll see the title, the wave emoji, the cards (with dashes), the headings... everything unstyled and ugly, but all there.

Try it yourself

Look at the page in the browser. Without styling it's a list of plain text. That's normal — the next step makes it pretty.

Step 4 — Style it (the dark ocean theme)

Goal

A `styles.css` that turns the bare HTML into the dark, ocean-themed dashboard you saw in the screenshot.

Do it

PROMPT FOR CLAUDE CODE

Write the file `styles.css` for a dark “ocean” themed dashboard. Use:

1. A `:root` block with CSS variables: `--bg: #061826`, `--bg-soft: #0b2538`, `--panel: #0f2e44`, `--line: #1d4d6e`, `--text: #eaf6ff`, `--muted: #8fb6cf`, `--accent: #2bd9c4`, `--accent-2: #3aa0ff`, `--warn: #ffd166`, `--radius: 16px`, and a `--shadow` like `0 10px 30px rgba(0,0,0,0.35)`.
2. `body` with the `--text` color, a system-ui font, and a background that combines two `radial-gradient`s in blue/turquoise on top of `--bg`.
3. `.site-header`: flex, space-between, sticky-feeling with `backdrop-filter: blur(6px)`, bottom border in `--line`. The 🌊 emoji is big with a soft drop-shadow in the accent color. The word “Sustainability Lab” inside the `<h1>` is in `--accent`.
4. `.container`: max-width ~1100px, centered, padded.
5. `.status-bar`: small, horizontal, with a colored `.dot` (gray when loading, turquoise `--accent` when `.ok`, red `#ef4444` when `.err`).
6. `.cards`: a CSS grid of 3 columns on desktop, 1 on mobile (use `repeat(auto-fit, minmax(220px, 1fr))`). Each `.card` is a rounded panel using `--panel` and `--shadow`, with a large `.big` number and a small `.unit` next to it.
7. `.day-nav`: horizontal flex with rounded `.day-btn` pills. The active pill (`.day-btn.active`) has the gradient `linear-gradient(135deg, var(--accent-2), var(--accent))`.
8. `.panel`: same rounded panel style as cards. Holds the chart and the table.
9. `.hourly table`: full width, comfortable padding, the `.now` row highlighted with a faint accent background. The `.wind-arrow` rotates via `transform`. The `.bar` is a small horizontal bar next to wave values, using a gradient.
10. Make it responsive – the cards collapse to one column on phones.

What you should see

Refresh. The page transforms: dark blue background, glowing wave emoji, the three cards laid out nicely (still with dashes), and clear typography.

Try it yourself

Ask Claude:

PROMPT FOR CLAUDE CODE

Make the accent color orange instead of turquoise. Change only the CSS variable.

You'll see every accent (the "Sustainability Lab" word, the active day pill, the chart line) turn orange. Change it back to `#2bd9c4` when you're done.

Step 5 — Data sources & the boot script

Goal

Start `app.js` : define where the data comes from (Open-Meteo) and write the boot function that fetches it when the page loads.

Do it

PROMPT FOR CLAUDE CODE

Open `app.js` and write the first part:

1. A `LOCATION` constant with: `name: "La Cícer Beach"`, `lat: 28.138`, `lon: -15.443`, `timezone: "Atlantic/Canary"`.
2. Two URL constants built with template strings:
 - `MARINE_URL` → `https://marine-api.open-meteo.com/v1/marine` with parameters `latitude`, `longitude`, `hourly=wave_height,sea_level_height_msl`, `timezone` (URL-encoded), and `forecast_days=4`.
 - `WEATHER_URL` → `https://api.open-meteo.com/v1/forecast` with `hourly=wind_speed_10m,wind_direction_10m` and the same location/ `timezone` parameters.
3. A `COMPASS` array with the 16 cardinal points: `N`, `NNE`, `NE`, `ENE`, `E`, `ESE`, `SE`, `SSE`, `S`, `SSW`, `SW`, `WSW`, `W`, `WNW`, `NW`, `NNW`.
4. A tiny `$` helper: `const $ = (id) => document.getElementById(id);`
5. Three module-scoped variables: `let HOURS = [];`, `let DAYS = [];`, `let activeDay = null;`
6. A `setStatus(kind, text)` function that updates the `#statusDot` class (add `ok` or `err`) and writes the message into `#statusText`.
7. An `async` function `init()` that:
 - Writes the location string into `#coords` (something like `La Cícer Beach · 28.138°N, 15.443°W · Atlantic/Canary`).
 - Calls `setStatus("loading", "Fetching live data from Open-Meteo...")`.
 - Uses `Promise.all` with `fetchJSON(MARINE_URL)` and `fetchJSON(WEATHER_URL)` to grab both responses in parallel.
 - On success: calls `setStatus("ok", "Live data · updated HH:MM ...")` with the current time.
 - On failure: catches the error and calls `setStatus("err", "Could not load data: ...")`.
8. A helper `async` function `fetchJSON(url)` that calls `fetch(url)`, throws if `!res.ok`, and returns `res.json()`.
9. At the bottom: `document.addEventListener("DOMContentLoaded", init);`

Don't implement `mergeData` or the renderers yet – leave them for the next steps.

What you should see

Refresh. The status dot should briefly be gray ("Fetching...") and then turn green ("Live data · updated ..."). If you open the browser console (F12), you should see no red errors. The cards still show dashes — that's next.

Try it yourself

Disconnect your wifi for a moment and reload. The status should turn red with an error message. Reconnect and reload.

Step 6 — Merge the two data feeds

Goal

Take the two JSON responses (marine + weather) and combine them into one tidy list, where each entry has the time, tide, wave, wind speed, and wind direction.

Do it

PROMPT FOR CLAUDE CODE

Add to `app.js` :

1. A `numOrNull(v)` helper that returns `null` if `v` is `null/undefined/NaN`, otherwise `Number(v)`.
2. A `mergeData(marine, weather)` function that:
 - Reads `marine.hourly.time`, `marine.hourly.sea_level_height_msl`, `marine.hourly.wave_height`.
 - Reads `weather.hourly.time`, `weather.hourly.wind_speed_10m`, `weather.hourly.wind_direction_10m`.
 - Builds a `Map` from each weather time to `{ windSpeed, windDir }`.
 - Maps each marine time `t` to an object: `{ time: t, date: new Date(t), day: t.slice(0,10), hour: new Date(t).getHours(), tide, wave, windSpeed, windDir }`, looking up the matching wind from the map (or `null` if missing).
 - Returns the array.
3. Update `init()` so that after the two fetches succeed it stores the result: `HOURS = mergeData(marine, weather);` and throws if `HOURS.length === 0`. Then computes `DAYS = [...new Set(HOURS.map(h => h.day))];` and sets `activeDay = dayKey(new Date())` (using a helper, see next).
4. Add three small helpers: `compass(deg)` returns `COMPASS[Math.round(deg/22.5) % 16]`; `fmt(v, dp)` returns `"-"` if `null` else `v.toFixed(dp)`; `dayKey(d)` returns `YYYY-MM-DD` from a `Date`; `dayLabel(key)` returns a short label like `"Thu 11"` using `toLocaleDateString("en-GB", { weekday: "short", day: "numeric" })`.
5. Add `waveWord(m)`: returns `"Flat"` if `<0.3`, `"Small"` `<0.6`, `"Knee-waist high"` `<1.0`, `"Chest high"` `<1.5`, `"Overhead"` `<2.5`, otherwise `"Big"`.

What you should see

The page still looks the same — we're only preparing the data. Open the browser console and type `HOURS.length` → it should print a number around 96 (24 hours × 4 days). Type `HOURS[0]` → you'll see one row of merged data.

Try it yourself

In the console, type `DAYS` to see the four upcoming days, and `HOURS.find(h => h.tide > 1)` to find the first hour with a tide above 1 meter.

Step 7 — Current conditions cards

Goal

Fill the three cards (Tide / Wind / Waves) with the values of **the hour closest to now**.

Do it

PROMPT FOR CLAUDE CODE

Add to `app.js` a `renderCurrent()` function that:

1. Finds the entry in `HOURS` whose `date` is closest to `Date.now()`. Call it `cur`.
2. Writes the values into the DOM:
 - `$("#curTide").textContent = fmt(cur.tide, 2);`
 - `$("#curWind").textContent = fmt(cur.windSpeed, 0);`
 - `$("#curWave").textContent = fmt(cur.wave, 2);`
3. If `cur.windDir` is not null: write `${compass(cur.windDir)} (${Math.round(cur.windDir)}°)` into `#curWindDir`, and set `$("#curWindArrow").style.transform = "rotate(" + cur.windDir + "deg)"`.
4. Tide trend: look at the next hour (`HOURS[indexOf(cur) + 1]`). If its tide is greater, write `"rising ▲"` into `#curTideTrend`; if less, `"falling ▼"`.
5. If `cur.wave` is not null: write `${waveWord(cur.wave)} · significant height` into `#curWaveSub`.

In `init()`, call `renderCurrent()` after the data is loaded.

What you should see

The three cards now show real numbers: the current tide in meters, the wind speed in km/h with a rotating arrow, and the wave height with a description like “Chest high · significant height”.

Try it yourself

Ask Claude:

PROMPT FOR CLAUDE CODE

Make the wind arrow bigger and add a CSS transition so it animates smoothly when the value updates.

Step 8 — Day selector

Goal

A row of buttons (Today, Fri 12, Sat 13, Sun 14) that lets you pick which day's data to look at.

Do it

PROMPT FOR CLAUDE CODE

Add to `app.js` :

1. `renderDayNav()` that:
 - Clears `#dayNav.innerHTML` .
 - For each `day` in `DAYS` , creates a `<button class="day-btn">` . Its label is "Today" if `day === dayKey(new Date())` , otherwise `dayLabel(day)` .
 - Adds the class `active` to the one that matches `activeDay` .
 - On click: set `activeDay = day` , remove `active` from all buttons, add `active` to the clicked one, then call `renderDay(day)` .
 - Appends each button to `#dayNav` .
2. `renderDay(day)` that:
 - Filters `HOURS` to `rows = HOURS.filter(h => h.day === day)` .
 - Calls `drawTideChart(rows)` (Step 9) and `renderTable(rows)` (Step 10) – leave the calls in place even though the functions don't exist yet.
 - Computes the low and high tide of those rows and writes into `#tideRange` : `low X.XX m · high Y.YY m` .

In `init()` , call `renderDayNav()` and `renderDay(activeDay)` after `renderCurrent()` .

What you should see

A row of four pills appears under the cards. The first ("Today") is highlighted in turquoise. Clicking another one moves the highlight. The browser console will show errors about `drawTideChart` / `renderTable` not being defined — that's fine; we add them next.

Try it yourself

Click the buttons. Notice how only the highlight changes for now.

Step 9 — Tide chart on a canvas

Goal

A custom line chart drawn on a `<canvas>` showing the tide curve hour by hour for the selected day.

Do it

PROMPT FOR CLAUDE CODE

Add to `app.js` a function `drawTideChart(rows)` that draws on the `<canvas id="tideChart">`. Use the 2D context, no chart library.

1. Resize the canvas for crisp rendering: get `dpr = window.devicePixelRatio || 1`; set `canvas.width = cssW * dpr` and `canvas.height = 220 * dpr` (where `cssW = canvas.clientWidth || 800`); then `ctx.setTransform(dpr, 0, 0, dpr, 0, 0)` and `clearRect`.
2. Filter `rows` to those where `tide != null` → `pts`. If fewer than 2 points, return.
3. Compute padding (`padL=40, padR=16, padT=16, padB=28`), the inner width `w` and height `h`, and the value range `[min, max]` of the tide values. Add a small visual padding to the range. Define helpers `x(i)` (horizontal position) and `y(v)` (vertical position).
4. Draw a faint horizontal grid (4 lines) with Y-axis labels in `--muted color (#8fb6cf)`. Use `ctx.font = "11px system-ui, sans-serif"`.
5. Draw X-axis labels every 3 hours (`"00h", "03h", ...`).
6. Fill the area under the curve with a vertical gradient from `rgba(58,160,255,0.35)` at the top to `rgba(58,160,255,0.02)` at the bottom.
7. Draw the line on top: `strokeStyle = "#3aa0ff", lineWidth = 2.5`.
8. If the day shown is today, draw a turquoise dot (`#2bd9c4`) at the current hour with a dark outline.

Also: add a window resize listener that, if `activeDay` is set, recalls `drawTideChart(HOURS.filter(h => h.day === activeDay))`.

What you should see

A smooth curve of the tide for the selected day, with hour labels along the bottom and meter values on the left. If you're viewing today, a turquoise dot marks the current hour. Switching days redraws the chart.

Try it yourself

Resize the browser window. The chart should redraw to fit.

Step 10 — Hourly table

Goal

A table with one row per hour of the selected day, showing tide, wind direction + speed, and wave height with a small bar.

Do it

PROMPT FOR CLAUDE CODE

Add to `app.js` a function `renderTable(rows)` that:

1. Clears `#hourlyBody.innerHTML`.
2. Computes `nowHour = new Date().getHours()`, `today = dayKey(new Date())`, and `maxWave = Math.max(0.1, ...rows.map(r => r.wave ?? 0))`.
3. For each `r` in `rows`, creates a `<tr>`. If `r.day === today` && `r.hour === nowHour`, sets `tr.className = "now"`.
4. Each row's HTML has four cells:
 - Hour as `"HH:00"`.
 - Tide formatted with `fmt(r.tide, 2)`.
 - Wind cell: an arrow `<span class="wind-arrow" style="transform:rotate(${r.windDir ?? 0}deg)">↑</span>`, then `${fmt(r.windSpeed, 0)} km/h`, then a `<span class="dir-label">` with the compass letters.
 - Wave: `${fmt(r.wave, 2)}` plus a `<span class="bar">` whose width in px is `Math.round((r.wave / maxWave) * 60)`.

Make sure `renderDay()` (from Step 8) is calling `renderTable(rows)` — it should already.

What you should see

Below the chart, a clean table appears with 24 rows. The row for the current hour is highlighted. The wind arrows point in different directions. The little bars next to wave heights give a quick visual sense of which hour has the biggest waves.

You're done! 🎉

Try it yourself

Ask Claude:

PROMPT FOR CLAUDE CODE

Add a fifth column called “Surf?” that shows a 🏄 emoji if the wave is between 0.6 and 1.5 meters, and a 😞 emoji otherwise.

Make it yours

You have a working app. Now make it yours:

Change the beach. Open `app.js` and modify the `LOCATION` constant with the latitude and longitude of your favorite beach. You can grab them from Google Maps (right-click on a point → copy coordinates).

PROMPT FOR CLAUDE CODE

Change `LOCATION` in `app.js` to point to Las Canteras Beach (latitude 28.142, longitude -15.439, timezone "Atlantic/Canary").

Change the colors. Edit the `:root` block in `styles.css` — change the `--accent` and `--accent-2` variables to warm tones (orange, pink).

PROMPT FOR CLAUDE CODE

Change the theme colors to a sunset palette: keep the dark background, but change `-accent` to a warm orange and `-accent-2` to a soft pink.

Add a new data field. Open-Meteo has many more fields: water temperature, UV index, rain.

PROMPT FOR CLAUDE CODE

Add the UV index to the dashboard. You need to: 1. Add `uv_index` to the `hourly=` parameters in `WEATHER_URL`. 2. Read it in `mergeData()` and add it as a `uv` field on each row. 3. In `index.html`, add a fourth card alongside Tide/Wind/Waves with `id="curUv"`. 4. In `renderCurrent()`, write the value into `#curUv`.

When things break

What you see	Ask Claude
The page is blank	“The page is blank. Open <code>index.html</code> and check the structure for an error.”
“Failed to fetch” in the browser console	“The browser console says ‘Failed to fetch’. Check if I’m online and verify the URLs in <code>app.js</code> .”
Status dot stays gray forever	“The status dot stays on ‘loading’. Look at <code>init()</code> in <code>app.js</code> and check whether the fetch is completing.”
Cards still show “—”	“The cards aren’t getting filled. Make sure <code>renderCurrent()</code> is called inside <code>init()</code> after the data loads.”
“Port 8000 is already in use”	“Port 8000 is busy. Start the Python server on port 8001 instead.”
<code>python3: command not found</code>	“The <code>python3</code> command isn’t found. Help me check whether Python 3 is installed and on the PATH.”
Chart canvas is empty	“The tide chart is blank. Open the console, look for an error in <code>drawTideChart</code> , and fix it.”

Golden rule: copy the full error message from the terminal or browser console, and paste it to Claude. Half the time, the fix comes in 30 seconds.

What's next

Put your app online (free, in 1 minute) — your app is just three static files, so you can drop them anywhere:

- **Netlify Drop:** open <https://app.netlify.com/drop>, drag your `surf-api` folder into the page. Done — you get a public URL.
- **GitHub Pages:** push the folder to a GitHub repo, enable Pages in repo settings.
- **Vercel CLI:** from the project folder, run `npx vercel` (needs Node.js).

Join the Agüita community:

- WhatsApp: <https://wa.me/34603786656>
- Come to the next workshop. Bring a friend.

Keep learning:

- <https://developer.mozilla.org/en-US/docs/Learn> — MDN's tutorials for HTML, CSS, and JavaScript.
 - <https://open-meteo.com/en/docs> — all the free weather data you can request.
 - <https://docs.claude.com/claude-code> — every Claude Code trick.
-

Glossary

- **HTML** — the language that describes the *structure* of a web page (headings, paragraphs, images, etc.).
 - **CSS** — the language that describes *how it looks* (colors, fonts, layout).
 - **JavaScript** — the language that *makes it do things* (reacts to clicks, fetches data, updates the screen).
 - **DOM** — “Document Object Model”. The tree of elements the browser builds from your HTML. JavaScript reads and changes it via `document.getElementById(...)` and similar.
 - **API** — a service on the internet you ask for data. Open-Meteo is an API.
 - **Endpoint** — the exact URL of an API. `MARINE_URL` is an endpoint.
 - **JSON** — the format APIs return data in. It’s like a nested list of “name: value” pairs.
 - **fetch** — the JavaScript function that asks an API for data. Returns a *Promise*.
 - **Promise** — JavaScript’s way of saying “I’ll have the answer in a moment”. `async / await` lets you wait for it.
 - **Canvas** — an HTML element you can draw on pixel by pixel with JavaScript. Our tide chart uses one.
 - **Prompt** — the instruction you give Claude Code. The more specific, the better the result.
-

Thanks for coming to the workshop.

Agüita House · Sustainability Lab · 2026